

Aufgabe 1: AVL-Bäume**(10 Punkte)**

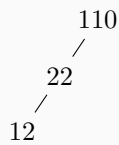
Fügen Sie die folgenden Zahlen nacheinander in einen *AVL-Baum* ein:

110 22 12 33 35 8

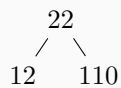
Zeichnen Sie den Baum vor und nach jeder durchgeführten Rotation. Geben Sie auch jeweils an, was für Rotationen Sie durchführen.

Lösung

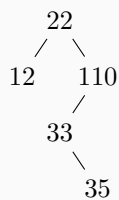
Einfügen von 110, 22, 12



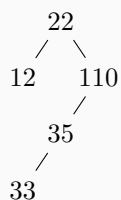
R-Rotation um 110



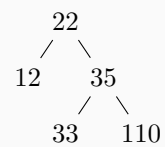
Einfügen von 33, 35



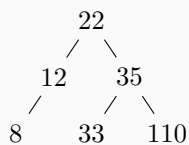
LR-Rotation um 110 (L)



LR-Rotation um 110 (R)



Einfügen von 8



Aufgabe 2: Heaps**(10 Punkte)**

Fügen Sie die folgenden Zahlen nacheinander in einen *Min-Heap* ein:

70 12 30 8 10 1 45

Zeichnen Sie den Baum vor und nach jedem vollständigen Einfügen.

Lösung**Einfügen von 70**

70

Einfügen von 12

12
/ 70

Einfügen von 30

12
/ 70 \ 30

Einfügen von 8

8
/ 12 \ 30
/ 70

Einfügen von 10

8
/ 10 \ 30
/ 70 \ 12

Einfügen von 1

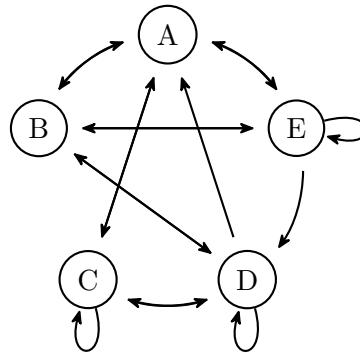
1
/ 10 \ 8
/ 70 \ 12 \ 30

Einfügen von 45

1
/ 10 \ 8
/ 70 \ 12 \ 30 \ 45

Aufgabe 3: Graphen**(10 Punkte)**

Betrachten Sie den folgenden gerichteten Graphen:



- Erstellen Sie die Adjazenzliste und die Adjazenzmatrix zu diesem Graphen.
- Welche der beiden Darstellungen würden Sie wählen, wenn Sie ...
 - ... häufig überprüfen müssen, ob eine Kante zwischen zwei Knoten existiert?
 - ... häufig über alle Nachbarn eines Knotens iterieren müssten?

Lösung**Adjazenzliste:**

- A: B, C, E
- B: A, D, E
- C: A, C, D
- D: A, B, C, D
- E: A, B, D, E

Adjazenzmatrix:

	A	B	C	D	E
A	0	1	1	0	1
B	1	0	0	1	1
C	1	0	1	1	0
D	1	1	1	1	0
E	1	1	0	1	1

Effizienz: Da der Graph relativ dicht ist, d.h. viele Kanten enthält, kann die Adjazenzmatrix eine gute Wahl sein, um schnell zu überprüfen, ob eine Kante zwischen zwei Knoten existiert. In diesem Fall ist der Speicher-Overhead im Vergleich zu einer Adjazenzliste nur gering, der Performance-Vorteil beim Zugriff aber deutlich.

Wenn es jedoch darum geht, über alle Nachbarn eines Knotens zu iterieren, ist die Adjazenzliste die bessere Wahl, da sie nur die vorhandenen Kanten speichert. Bei der Adjazenzliste müsste beim Iterieren jedes Mal unterschieden werden, ob eine Kante existiert oder nicht, was bei einer Adjazenzliste wegfällt.

Aufgabe 4: Datenstrukturen**(10 Punkte)**

Erklären Sie Idee und Funktionsweise des folgenden Datentyps. Welche Vor- und Nachteile sehen Sie gegenüber einem binären Suchbaum?

```
1  type Table struct {
2      keys    []string
3      values  []string
4  }
5
6  func targetIndex(key string) int {
7      firstchar := key[0]
8      return int((firstchar-'a') * 10
9  }
10
11 func (h *Table) FirstEmptyIndexForKey(key string) int {
12     for i := 0; i < 260; i++ {
13         index := (targetIndex(key) + i) % 260
14         if h.keys[index] == "" {
15             return index
16         }
17     }
18     return -1
19 }
20
21 func (h *Table) FirstMatchingIndexForKey(key string) int {
22     for i := 0; i < 260; i++ {
23         index := (targetIndex(key) + i) % 260
24         if h.keys[index] == key {
25             return index
26         }
27     }
28     return -1
29 }
30
31 func (h *Table) Put(key string, value string) {
32     index := h.FirstEmptyIndexForKey(key)
33     if index == -1 {
34         panic("Table is full")
35     }
36     h.keys[index] = key
37     h.values[index] = value
38 }
```

Lösung

Dieser Datentyp implementiert eine Tabelle, die Schlüssel auf Werte (beides Strings) abbildet. Damit erfüllt sie einen ähnlichen Zweck wie ein binärer Suchbaum.

Funktionsweise: Die Tabelle kann 260 Schlüssel-Wert-Paare speichern. Die Idee ist, dass für jeden Buchstaben im Alphabet 10 Plätze vorhanden sind. Beim Einfügen eines neuen Schlüssels wird anhand des ersten Buchstabens die Zielposition bestimmt. Falls diese Position bereits belegt ist, wird der nächste Platz in der Liste verwendet.

Vor- und Nachteile:

- Im Idealfall ist die Suche sehr schnell, da die Position immer direkt berechnet werden kann.

- Im Worst-Case muss allerdings die gesamte Liste durchlaufen werden, was sehr langsam ist.
- D.h. im Worst-Case ist die Suche mit $O(n)$ schlechter als bei einem binären Suchbaum. Solange die Liste nicht zu voll ist, ist die Suche aber im Durchschnitt schneller.

Anmerkung: Diese Art Datenstruktur nennt man eine *Hashtabelle*. Sie ist eine der wichtigsten Datenstrukturen in der Informatik. Sie erreicht im Durchschnitt eine bessere Laufzeit als ein binärer Suchbaum, allerdings erkauft man sich diesen Vorteil mit einem stark erhöhten Speicherbedarf.

Aufgabe 5: Komplexität**(10 Punkte)**

Bestimmen Sie die Komplexität des folgenden Algorithmus zur Bestimmung des Medians einer Liste der Länge n : Geben Sie auch einen Verbesserungsvorschlag an, wie die Komplexität reduziert werden kann.

```
1 func Median(list []int) int {
2     diffs := make([]int, len(list))
3
4     for i, el := range list {
5         d := largerCount(list, el) - smallerCount(list, el)
6         diffs[i] = abs(d)
7     }
8
9     return list[smallestIndex(diffs)]
10 }
```

Gehen Sie dabei davon aus, dass die Hilfsfunktionen tun, was ihre Namen vermuten lassen:

- `largerCount` und `smallerCount` zählen die Anzahl der Elemente, die größer bzw. kleiner als ein gegebenes Element sind.
- `smallestIndex` gibt den Index des kleinsten Elements zurück.
- `abs` berechnet den Betrag einer Zahl.

Gehen Sie weiter davon aus, dass diese Hilfsfunktionen jeweils optimal sind, d.h. dass sie die bestmögliche Laufzeit für ihre Aufgabe haben.

Lösung**Analyse:**

- Die Schleife wird n -mal durchlaufen.
- Dabei werden `largerCount`, `smallerCount` und `abs` jeweils einmal aufgerufen.
- Die Hilfsfunktion `smallestIndex` wird einmal mit einer Liste der Länge n aufgerufen.
- Die Listen-Hilfsfunktionen können jeweils eine Laufzeit von $O(n)$ haben.
- Die Funktion `abs` hat eine Laufzeit von $O(1)$, ist also vernachlässigbar.

Insgesamt ergibt sich eine Komplexität von $O(n)$ im Schleifenrumpf und damit $O(n^2)$ für den gesamten Algorithmus.

Verbesserungsvorschlag:

Wenn man die Liste zu Beginn der Median-Funktion sortiert, kann der Median anschließend in konstanter Zeit bestimmt werden, nämlich als das Element an der Position $\frac{n}{2}$.

Die Sortierung hat eine Komplexität von $O(n \log n)$, so dass die Bestimmung des Medians ebenfalls in $O(n \log n)$ möglich ist.